

APIs de IA y Diseño de Interfaces

Módulo 9 · Infraestructura y Despliegue

Estandar OpenAI, function calling, streaming, rate limiting, versionado y el protocolo MCP para tool use.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

APIs de IA y Diseño de Interfaces

La API es la interfaz entre el sistema LLM y sus consumidores: las aplicaciones que lo usan. Un diseño de API deficiente produce: sistemas que son difíciles de usar correctamente, integraciones frágiles que se rompen con cada actualización, problemas de seguridad por autenticación o autorización mal implementada, y dificultades para evolucionar el sistema sin romper la compatibilidad. Un diseño de API sólido es tan importante como la calidad del modelo subyacente.

En 2025, el estándar de facto para APIs de LLM es la API de OpenAI. El endpoint `/v1/chat/completions` con su formato de mensajes (`system/user/assistant`) y sus parámetros (`model`, `temperature`, `max_tokens`, `stream`) se ha convertido en el protocolo universal que vLLM, TGI, Ollama, y casi todos los frameworks de inferencia implementan. Esto significa que el código que funciona con la API de OpenAI generalmente funciona también con un servidor vLLM autohosteado simplemente cambiando la base URL.

El estándar de API OpenAI y sus alternativas

El formato de API OpenAI chat completions

El formato estándar de chat completions de OpenAI envía un array de mensajes con roles (`system`, `user`, `assistant`) y recibe la respuesta del modelo con el token usage y los finish reason. Los parámetros clave son: `model` (el modelo a usar), `messages` (el historial de conversación), `temperature` (0-2, controla la aleatoriedad), `max_tokens` (límite de tokens de respuesta), `stream` (`true` para streaming), `response_format` (para JSON mode), y `tools` (para function calling). Este formato es el que implementan vLLM (`python -m vllm.entrypoints.openai.api_server`), TGI (v2+ con modo OpenAI-compatible), Ollama, y LM Studio.

Function calling / Tool use

Function calling permite al LLM indicar que necesita ejecutar una función y proporciona el nombre y los argumentos. El flujo es: el cliente envía la petición con la lista de herramientas disponibles (`tools`) y sus schemas JSON. El modelo decide si usar una herramienta y responde con un `tool_call` que especifica qué función llamar y con qué argumentos. El cliente ejecuta la función y envía el resultado como un mensaje con role `tool`. El modelo usa el resultado para continuar la conversación. Este mecanismo es la base de los sistemas agénticos (M10).

Streaming: entrega progresiva de tokens

El streaming (`stream: true`) devuelve los tokens de la respuesta a medida que se generan en lugar de esperar a que la respuesta esté completa. Los eventos SSE (Server-Sent Events) llegan con el formato data: `{"choices": [{"delta": {"content": "token"}}]}`. El streaming reduce drásticamente la latencia percibida por el usuario: en lugar de esperar 10 segundos a que se complete una respuesta larga, el usuario ve los primeros tokens en <1 segundo. Es imprescindible para aplicaciones conversacionales.

Principios de diseño de API para sistemas LLM

Versionado de API

Las APIs de sistemas LLM evolucionan con los modelos y las funcionalidades. El versionado de API garantiza que las integraciones existentes no se rompen cuando el sistema evoluciona. Las estrategias de versionado son: versión en la URL (`/v1/`, `/v2/`), versión en el header (`API-Version: 2024-11-20`), y versión en el parámetro del modelo (donde el modelo es el contrato: `gpt-4o-2024-11-20` siempre se comporta igual). OpenAI usa una combinación: URLs versionadas (`/v1/`) con snapshots de modelo fechados.

Rate limiting y quotas

El rate limiting protege el sistema de abuso y garantiza una distribución equitativa de recursos. Los tipos de límites: RPM (requests per minute), TPM (tokens per minute), y simultaneous requests. La implementación correcta devuelve HTTP 429 con headers `Retry-After` cuando se exceden los límites, y los clientes implementan backoff exponencial para reintentos. Para sistemas multi-tenant, los límites se configuran por cliente (API key) y por plan.

Diseño de la respuesta de error

Las APIs de LLM deben devolver errores estructurados que permitan a los clientes distinguir entre errores de cliente (4xx), errores de servidor (5xx), y errores específicos del modelo (`finish_reason = "length"` cuando la respuesta se trunca por `max_tokens`, `"content_filter"` cuando el contenido es filtrado). La respuesta de error debe incluir un código de error machine-readable, un mensaje human-readable, y opcionalmente detalles adicionales para diagnóstico.

SECCIÓN 4 · EJEMPLOS REALES

- API interna de LLM para equipos de producto (empresa tecnológica): endpoint `/v1/chat/completions` compatible con OpenAI, con autenticación por API key rotada mensualmente, rate limiting de 100 RPM y 100K TPM por equipo, dashboard de uso por equipo para control de costes internos, y 30 días de retención de logs de peticiones para auditoría. Los equipos usan el mismo cliente OpenAI Python con `base_url` apuntando al servidor interno.
- API pública de LLM como producto (startup): gateway con Stripe para billing por tokens (webhook que actualiza el límite de quota cuando el cliente recarga crédito), rate limiting por tier (Free: 10 RPM/100K TPM, Pro: 100 RPM/1M TPM, Enterprise: custom), generación de API keys con hashing (Argon2) para no almacenar la clave en texto plano, y dashboard de uso para el cliente.
- Streaming en aplicación de chat empresarial: el cliente JavaScript usa el EventSource API del browser para recibir los tokens en streaming del servidor. El servidor proxy recibe el streaming de vLLM y lo reenvía al cliente añadiendo autenticación y logging. Latencia percibida: <500ms hasta el primer token vs 8-15s sin streaming.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Exponer el modelo directamente sin API Gateway. Sin API Gateway, el modelo no tiene autenticación, rate limiting, ni logging. Cualquier persona con acceso a la red puede usarlo ilimitadamente. El API Gateway es la capa de seguridad y control imprescindible entre los clientes y el modelo.

Error 2: No implementar streaming para aplicaciones conversacionales. Una respuesta de 500 palabras tarda 8-15 segundos en completarse sin streaming. Con streaming, el usuario ve los primeros tokens en <500ms. La diferencia en percepción de calidad del sistema es enorme. Streaming es obligatorio para cualquier aplicación conversacional.

Error 3: Rate limiting solo por RPM sin considerar los tokens. Un único request con un prompt de 100.000 tokens consume tanta GPU como 1.000 requests de 100 tokens. El rate limiting solo por RPM no protege de abusos con prompts masivos. Implementa siempre límites tanto por RPM como por TPM.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para implementar un API Gateway de LLM con FastAPI: define los endpoints (/v1/chat/completions, /v1/models, /v1/embeddings), implementa autenticación con API keys (middleware que verifica el header Authorization: Bearer api_key), implementa rate limiting con slowapi o similar, añade logging de cada request con httpx para forwarding al backend vLLM, y configura streaming con StreamingResponse para los requests con stream: true.

Para la gestión de API keys de forma segura: genera API keys con secrets.token_urlsafe(32) de Python (256 bits de entropía), almacena solo el hash (con Argon2 o bcrypt) en la base de datos, muestra la clave al usuario solo una vez en el momento de creación (igual que GitHub/OpenAI), y implementa rotación de claves sin downtime (acepta tanto la clave antigua como la nueva durante un periodo de transición).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M9-T2 (Arquitecturas de despliegue): el API Gateway es el componente de exposición de la arquitectura.
- M10 (Sistemas agénticos): el function calling y el protocolo MCP son la base de los sistemas agénticos.
- M12 (Seguridad): el diseño del API Gateway incluye la primera línea de defensa contra ataques y abusos.

SECCIÓN 8 · RESUMEN EJECUTIVO

El estándar de facto para APIs de LLM en 2025 es el formato OpenAI chat completions (/v1/chat/completions), implementado por vLLM, TGI, y Ollama. Los componentes clave del diseño de API son: autenticación (API keys con hash Argon2), rate limiting (por RPM y por TPM), versionado (URLs + snapshots de modelo), streaming (SSE, obligatorio para aplicaciones conversacionales), function calling (base de los sistemas agénticos), y gestión de errores estructurada. El protocolo MCP se convierte en estándar para tool use agéntico. Implementar un API Gateway propio añade autenticación, rate limiting, y logging antes de llegar al servidor vLLM.